AI-POWERED INDUSTRIAL ATTACHMENT MATCHING PLATFORM
(AISHA)

BIT 420 IT PROJECT II

PROJECT PROPOSAL

Prepared by:

Brian Ndinya

Joshua Omondi

Emmauel Muchina

Obadia Kipngetich

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

MASINDE MULIRO UNIVERSITY OF SCIENCE AND TECHNOLOGY

February 2026

DECLARATION

We declare that this project proposal is our original work and has not been presented for a degree award in any other university or institution. No part of this proposal may be reproduced without prior written permission of the authors and/or the institution.

NAME	REG/NO	SIGNATURE	DATE
Brian Ndinya	SIT/B/01-02627/2022		
Joshua Omondi	SIT/B/01-02916/2022		
Emmauel Muchina	SIT/B/01-02896/2022		
Obadia Kipngetich	SIT/B/01-02881/2022		

This proposal has been submitted with my approval as the University Supervisor.

Supervisor Name: _____

Signature: _____ Date: _____

Table of Contents

DECLARATION..... 2

ABSTRACT..... 5

CHAPTER ONE..... 6

BACKGROUND AND INTRODUCTION..... 6

1.1 Background..... 6

1.2 Statement of the Problem..... 7

1.3 Objectives..... 8

1.3.1 Main Objective..... 8

1.3.2 Specific Objectives..... 8

CHAPTER TWO..... 8

LITERATURE REVIEW..... 8

2.1 Introduction..... 8

2.2 Existing Placement Systems..... 9

2.3 Artificial Intelligence in Recruitment and Matching..... 9

2.4 E-Learning and Skill Development Platforms..... 10

2.5 Mobile Payment Systems in Africa..... 10

2.6 Identified Gaps..... 11

CHAPTER THREE..... 12

METHODOLOGY..... 12

3.1 Requirements Analysis and Specifications..... 12

3.1.1 Functional Requirements..... 12

3.1.2 Non-Functional Requirements..... 12

3.2 Conceptual Design..... 14

3.3 Functional Design.....15
3.3.1 Database Design.....15
3.3.2 AI/ML Algorithm Design.....15
3.3.3 User Interface Design.....16
3.4 Development Environment.....17
3.4.1 Hardware Requirements.....17
3.4.2 Software Requirements.....19
3.5 Development Process Design.....32
3.5.1 Choice of Methodology: Agile Scrum.....32
3.5.2 Software Development Life Cycle (SDLC) Stages.....32
3.5.3 Team Roles and Responsibilities.....32
3.6 Development Schedule.....33
3.6.2 Gantt Chart Representation (Visual Summary).....36
3.7 Development Budget.....36
REFERENCES.....40

ABSTRACT

The AI-Powered Industrial Attachment Matching Platform (AISHA) is an intelligent, automated platform designed to revolutionize the industrial attachment placement process for students, educational institutions, and companies. The system leverages Artificial Intelligence and Machine Learning technologies to automate matching between students and companies, eliminate corruption and favoritism, reduce administrative burden, and ensure optimal placements based on academic performance, skills, interests, and organizational needs. Key features include AI-powered matching with 90%+ accuracy, automated document generation (application letters, NITA forms, school letters), real-time tracking, intelligent chatbot support, payment integration for stipend management, and a comprehensive learning module for continuous skill development. The system addresses critical challenges such as manual application processes, limited opportunity visibility, skills-opportunity mismatch, and excessive administrative overhead. Built using free and open-source technologies including React, Node.js/Python, PostgreSQL, TensorFlow/PyTorch, and modern cloud services, the system is cost-effective, scalable, and sustainable. The implementation follows a phased- 14 week roadmap covering foundation, core development, AI/ML integration, advanced features, and testing/deployment phases, with expected outcomes including 90%+ student placement rates, 70% reduction in administrative time, and 99.9% system uptime.

CHAPTER ONE

BACKGROUND AND INTRODUCTION

1.1 Background

Industrial attachment is a critical component of technical and vocational education, providing students with practical work experience in their field of study. In Kenya, the National Industrial Training Authority (NITA) mandates that students in technical institutions undergo attachment training to bridge the gap between theoretical knowledge and practical skills required in the industry.

Despite its importance, the current attachment placement process faces numerous challenges. Students struggle to find suitable opportunities, often spending weeks or months in manual searches. The application process is cumbersome, requiring multiple documents including application letters, NITA registration forms, insurance certificates, and school letters detailing completed units. Educational institutions lack centralized systems to track student placements, while companies face difficulties in identifying qualified candidates from numerous applications.

These inefficiencies not only waste time and resources but also create opportunities for corruption and favoritism. Students without connections often miss out on quality placements despite having excellent academic records. The lack of transparency in the placement process undermines the fairness and effectiveness of industrial attachment programs.

Recent advancements in Artificial Intelligence and Machine Learning present an opportunity to transform this process. By automating matching algorithms, document generation, and application workflows, technology can eliminate human bias, reduce administrative overhead, and ensure that students are matched with opportunities that best suit their skills, interests, and career aspirations.

1.2 Statement of the Problem

The industrial attachment placement process in Kenya faces significant challenges that negatively impact students, educational institutions, and companies:

- For Students: Students experience difficulty in finding suitable attachment opportunities due to limited visibility into available positions. The manual application process is time-consuming, requiring preparation of multiple documents for each application. Corruption and favoritism in placement decisions result in unfair outcomes, with well-connected students securing better opportunities regardless of merit. There is often a mismatch between student skills/interests and assigned placements, leading to poor learning outcomes.
- For Educational Institutions: Institutions lack centralized systems to track student placements in real-time. Manual processing of attachment applications creates significant administrative overhead. There is limited data on placement success rates and student performance during attachments. Institutions struggle to maintain relationships with companies and ensure adequate placement opportunities for all students.
- For Companies/Organizations: Companies must manually screen numerous applications, which is time-consuming and inefficient. The lack of standardized information makes it difficult to assess student qualifications. Companies struggle to find students with specific skills required for their departments. The administrative burden of managing attachment programs discourages companies from participating.

These problems result in suboptimal outcomes for all stakeholders: students fail to secure appropriate placements, institutions cannot effectively manage their attachment programs, and companies miss out on talented students. Without a comprehensive solution, these inefficiencies will continue to undermine the quality and effectiveness of industrial attachment programs in Kenya.

1.3 Objectives

1.3.1 Main Objective

To design and develop an intelligent, automated Attachment Placement System that uses Artificial Intelligence and Machine Learning to optimize the matching of students with industrial attachment opportunities, eliminate corruption, reduce administrative burden, and improve placement success rates for all stakeholders.

1.3.2 Specific Objectives

1. To develop an AI-powered matching algorithm that achieves 85%+ accuracy in matching students with suitable attachment opportunities based on academic performance, skills, interests, and company requirements.
2. To automate the document generation process, reducing application preparation time by 70% through automatic creation of application letters, NITA forms, school letters, and insurance documentation.
3. To create a centralized web and mobile platform accessible to students (via mobile apps), institutions (via web and desktop apps), and companies (via web and desktop apps) with 99.9% uptime and response times under 2 seconds.
4. To implement a comprehensive learning module with personalized course recommendations that helps 70%+ of students improve their skills and increases placement success rates by 20%.
5. To integrate M-Pesa payment system for stipend management, enabling transparent and efficient financial transactions between companies and students.
6. To develop an intelligent chatbot with 50+ supported intents that provides 24/7 support and explains matching decisions to students, improving user satisfaction to 85%+.
7. To create real-time tracking and analytics dashboards that give institutions visibility into student placements and reduce their administrative time by 70%.
8. To ensure system security and compliance through end-to-end encryption, role-based access control, GDPR-ready data protection, and complete audit trails for all transactions.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing literature and systems related to student placement, AI-powered matching algorithms, and educational technology platforms. The review

examines similar systems, identifies gaps in current solutions, and establishes the theoretical foundation for the proposed system.

2.2 Existing Placement Systems

Several systems exist for managing student internships and attachments:

LinkedIn Talent Solutions: While LinkedIn provides a platform for job seekers and employers, it lacks specialized features for academic attachments, automated matching based on academic performance, or integration with institutional systems. The platform is general-purpose and does not address NITA compliance or Kenyan-specific requirements.

Handshake: Used primarily in the United States, Handshake connects students with employers but focuses on permanent employment rather than industrial attachments. It lacks AI-powered matching, automated document generation, and integration with African payment systems like M-Pesa.

WayUp: Another U.S.-based platform that connects students with internships. While it offers basic filtering, it does not provide intelligent matching, skill gap analysis, or learning recommendations.

These existing systems demonstrate the market need but fail to address the specific challenges of the Kenyan education system, including NITA integration, corruption elimination through AI-driven fairness, M-Pesa payment integration, and localized learning resources.

2.3 Artificial Intelligence in Recruitment and Matching

Research in AI-powered recruitment has demonstrated significant improvements in efficiency and accuracy:

Machine Learning Matching Algorithms: Studies have shown that gradient boosting algorithms (XGBoost, LightGBM) can achieve 85-90% accuracy in candidate-job matching when trained on features such as skills, experience, and performance metrics (Raghavan et al., 2020). These algorithms outperform traditional keyword-based matching by considering complex feature interactions.

Natural Language Processing: NLP techniques enable automatic extraction of skills and qualifications from resumes and job descriptions. BERT-based models have achieved state-of-the-art performance in intent classification and entity extraction (Devlin et al.,

2019), making them suitable for chatbot applications and document analysis.

Fairness in AI Systems: Research emphasizes the importance of fairness constraints in matching algorithms to prevent bias and ensure equal opportunity (Mehrabi et al., 2021). Techniques such as diversity injection and balanced sampling help mitigate discrimination in automated decision systems.

These findings support the proposed system's approach to using AI/ML for intelligent matching, document processing, and fairness-aware placement decisions.

2.4 E-Learning and Skill Development Platforms

The integration of learning recommendations with placement systems represents an innovative approach:

Personalized Learning: Research shows that personalized learning recommendations significantly improve student engagement and skill acquisition (Kizilcec et al., 2017). Content-based and collaborative filtering algorithms can effectively recommend courses based on student interests and career goals.

Skill Gap Analysis: Studies demonstrate that identifying and addressing skill gaps improves employment outcomes (Burning Glass Technologies, 2019). Systems that provide feedback on missing skills and recommend specific learning resources help students become more competitive.

The proposed system uniquely combines placement with skill development, creating a continuous improvement loop where students receive feedback, learn new skills, and improve their placement chances.

2.5 Mobile Payment Systems in Africa

Mobile money has transformed financial transactions in Africa:

M-Pesa Adoption: Kenya's M-Pesa system has over 50 million active users and processes billions of transactions annually (Safaricom, 2023). The Daraja API enables businesses to integrate M-Pesa payments into their applications, facilitating digital commerce.

Integration in Education: Studies show that mobile payment integration in educational systems improves transparency, reduces fraud, and accelerates disbursements (Jack & Suri, 2014). Digital stipend payments eliminate intermediaries and ensure students receive funds directly.

The proposed system leverages M-Pesa integration to modernize stipend management and provide transparent financial transactions.

2.6 Identified Gaps

The literature review reveals several gaps that the proposed system addresses:

- Lack of Kenyan-specific solutions that integrate NITA compliance and M-Pesa payments.
- Absence of AI-powered fairness mechanisms to eliminate corruption in placement processes.
- No existing systems combine placement with personalized skill development and learning recommendations.
- Limited automation in document generation for attachment-specific requirements.
- Inadequate support for multi-stakeholder platforms serving students, institutions, and companies simultaneously.
- Missing real-time tracking and analytics for institutional oversight.
- Insufficient attention to free and open-source solutions suitable for resource-constrained environments.

CHAPTER THREE

METHODOLOGY

3.1 Requirements Analysis and Specifications

3.1.1 Functional Requirements

The system will implement the following functional requirements:

- Student Module: Registration and profile management, AI-powered opportunity discovery with match scores, one-click application submission, real-time application tracking, document management, AI chatbot support, and learning recommendations.
- Institution Module: Student management with bulk import capabilities, real-time placement tracking dashboards, performance analytics, company relations management, API integration for portal synchronization, and automated reporting.
- Company Module: Opportunity posting with department specifications, automated student matching and ranking, application review and filtering, placement management, stipend configuration and M-Pesa integration, feedback and evaluation tools.
- Admin Module: Multi-tenant user management, institution and company verification, system configuration and feature flags, integration management, comprehensive audit logs, and platform-wide analytics.
- AI/ML Services: Intelligent matching engine with 85%+ accuracy, NLP-based document generation, chatbot with 50+ intent support, learning recommendation system, predictive analytics for placement success.
- Payment Integration: M-Pesa Daraja API integration for STK Push payments, stipend management and tracking, payment history and reconciliation, automated notifications for transactions.

3.1.2 Non-Functional Requirements

Performance: Page load time < 2 seconds, API response time < 500ms, support for 10,000+ concurrent users, real-time notifications with < 1 second delay.

Security: End-to-end encryption (AES-256), JWT authentication with refresh tokens, role-based access control, GDPR-ready data protection, comprehensive audit trails, regular security audits.

Scalability: Horizontal scaling capability, cloud-native microservices architecture, auto-scaling support, load balancing, database replication.

Availability: 99.9% uptime SLA, automated daily backups, disaster recovery plan, point-in-time recovery capability, redundancy for critical services.

Usability: Mobile-responsive design for all screen sizes, intuitive UI/UX following modern design principles, WCAG 2.1 accessibility compliance, comprehensive help documentation.

3.2 Conceptual Design

The system follows a microservices architecture with clear separation of concerns:

Presentation Layer: React-based web applications for all user types (students, institutions, companies, administrators), React Native mobile application for students and companies, Progressive Web App (PWA) capabilities for offline access.

API Gateway Layer: Centralized API gateway handling authentication, authorization, rate limiting, request routing, and load balancing. Implements JWT-based authentication with refresh token mechanism and role-based access control.

Application Services Layer: Microservices including User Service (authentication, profile management), Matching Service (AI-powered student-opportunity matching), Document Service (automated generation of letters and forms), Notification Service (email, SMS, push, WebSocket), Application Service (workflow management), Analytics Service (reporting and insights).

AI/ML Services Layer: Python-based services including Matching Engine (gradient boosting algorithm), NLP Engine (document processing), Chatbot AI (BERT-based intent classification), Learning Recommendation Engine (personalized course suggestions), Predictive Analytics (placement success prediction).

Data Layer: PostgreSQL for primary data storage, Redis for caching and session management, Supabase Storage for file storage, Vector database for chatbot knowledge base.

External Integrations: Institution portal APIs for student data synchronization, NITA system for form submission, M-Pesa Daraja API for payment processing, Email service (SendGrid), SMS service (Africa's Talking/Twilio).

3.3 Functional Design

3.3.1 Database Design

The database schema comprises 20+ tables organized into logical groups:

Core Tables: users, students, institutions, companies, departments, storing fundamental stakeholder information.

Placement Tables: opportunities, applications, placements, nita_forms, tracking the complete placement lifecycle.

Learning Tables: learning_resources, student_learning, student_skills, certifications, supporting the skill development module.

Financial Tables: payments, stipend_configs, managing payment transactions and configurations.

System Tables: notifications, messages, audit_logs, system_config, feature_flags, providing platform functionality and governance.

The schema implements proper normalization, foreign key constraints, indexes on frequently queried columns, and JSONB fields for flexible data storage.

3.3.2 AI/ML Algorithm Design

Matching Algorithm: The system uses gradient boosting (XGBoost/LightGBM) for student-opportunity matching. Features include normalized GPA, completed units (encoded), specialization similarity, location preference match, historical placement data, and company requirements. The model outputs match scores (0-100) with explanations and undergoes monthly retraining with new placement data.

Learning Recommendation: Content-based filtering combined with collaborative filtering recommends courses based on field of interest, current skill level, career goals, and learning history. Ranking considers relevance (40%), quality (25%), career alignment (20%), accessibility (10%), and peer completion rates (5%).

Chatbot NLP: BERT-based intent classification with 50+ supported intents including placement explanation, application help, status inquiry, and general FAQs. Entity extraction identifies key information from user queries. Response generation combines rule-based templates with GPT-powered natural language generation.

3.3.3 User Interface Design

The user interface follows modern design principles:

Design System: Chakra UI/TailwindCSS provides consistent, accessible components. Color scheme uses professional blues and greens representing trust and growth. Typography uses system fonts for optimal performance and readability.

Student Interface: Dashboard with personalized recommendations, opportunity cards with match scores, one-click application buttons, progress tracking timeline, learning module with course suggestions, chatbot accessible from all pages.

Company Interface: Opportunity management with drag-and-drop kanban boards, applicant cards with ranking and filtering, department management, analytics dashboards with charts, payment configuration forms.

Responsive Design: Mobile-first approach ensures optimal experience on all devices. Progressive enhancement provides advanced features on capable devices while maintaining core functionality on all platforms.

3.4 Development Environment

3.4.1 Hardware Requirements

Development Workstation Specifications

Based on the complexity of the AISHA system which runs multiple concurrent services (PostgreSQL database, Redis cache, Node.js backend, Python AI services, React frontend with hot module replacement, and mobile app emulators), the development workstation requires adequate computing resources to ensure smooth development experience without performance bottlenecks.

Component	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Core i5 (8th gen) or AMD Ryzen 5 3600, 4 cores, 8 threads	Intel Core i7 (10th gen+) or AMD Ryzen 7 5800X, 8 cores, 16 threads
Memory (RAM)	16GB DDR4 Allows running all services + IDE + browser	32GB DDR4 Smooth multitasking with multiple emulators
Storage	256GB SSD Fast read/write for node_modules and builds	512GB+ NVMe SSD Additional space for Docker images, databases
Display	Full HD (1920x1080) Sufficient for code editor + browser	2K/4K (2560x1440+) Better for multi-window development
Graphics	Integrated CPU (Intel UHD/AMD Radeon) Basic UI rendering	Dedicated GPU (NVIDIA/AMD) Faster Android emulator performance
Network	Ethernet or WiFi 5 (802.11ac) Stable internet for dependencies	Gigabit Ethernet Fast package downloads and API testing

Testing and Deployment Infrastructure

The AISHA mobile application is built using React Native with Expo framework, targeting both Android and iOS platforms. While emulators are suitable for development, physical device testing is essential to validate real-world performance, especially for features like push notifications, camera access (for profile pictures), and M-Pesa payment integration which may behave differently on actual devices.

Android Testing Device: Physical Android smartphone running Android 8.0+ for testing React Native mobile app. Devices like Samsung Galaxy A-series (KSH 20,000-30,000) or Xiaomi Redmi series (KSH 15,000-25,000) provide good value. Alternatively, Android Studio emulator can be used during development, though physical device testing is recommended before deployment.

iOS Testing Device (Optional): While the system targets both platforms, iOS testing requires Apple hardware (iPhone running iOS 13+) or access to MacOS for Xcode simulator. Given budget constraints and target market (Kenya where Android dominates with 80%+ market share), iOS testing is optional and can be deferred to later phases.

Multiple Screen Sizes: Testing across different screen sizes ensures responsive design works correctly. We utilize: (1) Small phones (5-5.5 inches) for minimum viewport testing, (2) Large phones (6+ inches) for typical modern devices, (3) Tablets (7-10 inches) for institution administrator use cases, (4) Desktop browsers at various resolutions (1366x768, 1920x1080, 2560x1440) for web portal testing. Browser Developer Tools' device emulation mode handles most testing scenarios without physical devices.

Production and Development Infrastructure

The AISHA system leverages cloud-native architecture with generous free tiers, minimizing infrastructure costs while maintaining production-grade reliability. Our infrastructure strategy prioritizes cost-effectiveness through strategic use of free and open-source platforms, as evidenced in the docker-compose.yml configuration and deployment scripts.

Service	Local Development	Production Platform	Purpose in AISHA
Frontend (Web)	Vite dev server (localhost:5173)	Vercel / Netlify Free tier services	Hosts React 19 web application for students, institutions, companies, and admins
Backend API	Node.js + Express (localhost:3000)	Railway / Render Free tier	RESTful API handling authentication, CRUD operations, file uploads, and WebSocket

			connections
AI/ML Services	Python FastAPI (localhost:8001)	Railway / Render Free tier	Handles student-opportunity matching algorithm, learning recommendations, and document generation
PostgreSQL Database	Docker container postgres:15-alpine	Supabase Free 500MB	Primary database storing users, students, companies, opportunities, applications, and all relational data
Redis Cache	Docker container redis:alpine	Upstash Redis Free 10k requests/day	Manages Bull job queues for background tasks, WebSocket session storage, and caching frequently accessed data
File Storage	Local filesystem backend/uploads	Supabase Storage Free 1GB	Stores uploaded documents (CVs, recommendation letters, NITA forms), profile pictures, and generated PDFs
CDN & Security	N/A (local)	Cloudflare Free tier	Provides DDoS protection, SSL/TLS certificates, edge caching for static assets, and DNS management

3.4.2 Software Requirements

Core Development Tools

Our choice of development tools is driven by actual project needs identified through codebase analysis. The AISHA project uses TypeScript extensively across frontend and

backend (as seen in package.json files), requiring robust IDE support with intelligent code completion and type checking.

Tool	Version / Details	Usage in AISHA Project
Visual Studio Code	Latest (v1.85+) Extensions: ESLint, Prettier, TypeScript, Docker, PostgreSQL	Primary IDE for TypeScript/JavaScript development. Used across frontend (React), backend (Node.js), and mobile (React Native) codebases. ESLint integration ensures code quality matching our .eslintrc configurations.
Git	V2.40+ Must support LFS for large files	Version control for entire monorepo structure. Configured with Git-flow branching strategy (main, develop, feature branches). Required for GitHub Actions CI/CD pipeline defined in .github/workflows.
Docker Desktop	v24.0+ with Docker Compose v2.20+	Manages local development environment as defined in docker-compose.yml. Runs PostgreSQL 15 (postgres:15-alpine image), Redis (redis:alpine), backend service, and AI service in isolated containers with shared saps_network bridge network.
Node.js	v20.11.0 LTS or higher	JavaScript runtime for frontend (Vite build tool), backend (Express 5.2.1 server), and mobile (Expo CLI). Version 20+ required for native fetch API used in apiClient.ts and for optimal performance with Vite

		7.2.4.
Python	v3.11+ (not 3.13, compatibility issues)	Powers AI/ML services using FastAPI 0.115.0. Runs matching algorithms, NLP for chatbot, and PDF generation. Python 3.11 chosen for performance improvements and required by scikit-learn 1.5.2 and TensorFlow dependencies.
Postman / Insomnia	Latest version	API testing for 130+ backend endpoints (students, opportunities, applications, payments, etc.). Import OpenAPI/Swagger specs auto-generated by FastAPI for AI service testing.
pgAdmin / DBeaver	pgAdmin 4 or DBeaver CE	Database management and query tool for PostgreSQL. Used to inspect 30+ tables schema, run migrations, debug data relationships, and optimize indexes on frequently queried columns (students.id, applications.status).

Frontend Web Application Stack

The web frontend serves four distinct user portals (student, institution, company, admin) with role-based routing. Analysis of frontend/package.json and frontend/src code reveals a modern React setup with TypeScript for type safety, Chakra UI v3 for component library, Redux Toolkit for state management, and Vite for lightning-fast builds.

React 19.2.0: Latest React with automatic batching and improved concurrent features. Used extensively in all pages (StudentDashboard.tsx, CompanyDashboard.tsx, etc.). React 19 provides better performance for our data-heavy dashboards.

TypeScript 5.9.3: Strongly typed JavaScript ensuring type safety across 50+ components. All .tsx files use TypeScript interfaces (types/student.ts, types/application.ts, types/opportunity.ts) preventing runtime errors and improving developer experience.

Vite 7.2.4: Next-generation build tool replacing webpack. Provides instant hot module replacement during development and optimized production builds. Configured in vite.config.ts with React plugin and path aliasing for cleaner imports.

Chakra UI 3.32.0: Component library providing 100+ accessible UI components (Box, Grid, Card, Table, Modal). Used throughout the application for consistent design. Installed with @emotion/react and @emotion/styled for CSS-in-JS styling.

Redux Toolkit 2.11.2: State management library handling global application state. Implemented in 5 slices: studentSlice (profile, applications, opportunities), companySlice (applicants, placements), authSlice (authentication), adminSlice (user verification), and institutionSlice (student management). Configured in store/index.ts with TypeScript support.

React Router DOM 7.13.0: Client-side routing library enabling navigation between student portal (/student/*), company portal (/company/*), institution portal (/institution/*), and admin portal (/admin/*). Implements protected routes with authentication checks.

Axios 1.13.4: HTTP client for API requests. Configured in services/apiClient.ts with base URL, authentication interceptors, and automatic token refresh. Used in all service files (studentService.ts, companyService.ts, paymentService.ts).

Formik 2.4.9 + Yup 1.7.1: Form management (Formik) and validation (Yup) libraries. Handles complex forms like student profile editing, opportunity posting, and application submission with client-side validation before API calls.

Framer Motion 12.33.0: Animation library for smooth UI transitions. Used for page transitions, modal animations, and interactive components enhancing user experience especially in dashboards and opportunity cards.

React Icons 5.5.0: Icon library providing 10,000+ icons from popular sets (FontAwesome, Material Design). Used throughout UI for intuitive visual indicators (FiUser, FiBriefcase, FiFileText etc.).

ESLint 9.39.1: Code linting tool configured with TypeScript and React rules. Enforces code quality standards, catches potential bugs, and ensures consistent code style across the team. Configured with react-hooks and react-refresh plugins.

Backend API Server Stack

The Node.js backend (backend/package.json) uses Express 5 for routing, PostgreSQL via pg driver for database operations, Redis for job queue management with Bull, Socket.io for real-time notifications, and PDFKit for automated document generation. All backend code is written in TypeScript (backend/src/*.ts) compiled to JavaScript for production.

Express 5.2.1: Fast, minimalist web framework for Node.js. Handles all HTTP routing for 130+ API endpoints organized in routes/ directory (studentRoutes.ts, opportunityRoutes.ts, applicationRoutes.ts, paymentRoutes.ts). Express 5 provides better middleware composition and native async/await support.

TypeScript 5.9.3 + ts-node: Backend is fully TypeScript for type safety. Uses interfaces for request/response types, database models, and service contracts. Compiled with tsc during builds, run with ts-node during development via nodemon.

PostgreSQL Driver (pg) 8.18.0: Official PostgreSQL client for Node.js. Connects to PostgreSQL database (localhost:5432 in development, Supabase in production). Executes parameterized queries preventing SQL injection. Used in services for CRUD operations on 30+ tables.

Bull 4.16.5: Redis-based queue library for handling background jobs. Manages asynchronous tasks like sending emails after application submission, generating documents (NITA forms, application letters), processing M-Pesa payment callbacks, and running nightly placement analytics. Queues defined in jobs/ directory with separate workers.

Redis 5.10.0: In-memory data structure store. Used both as Bull job queue backend and for caching frequently accessed data (active sessions, student profiles during matching). Connects to Redis container (localhost:6379) managed by docker-compose.yml.

Socket.io 4.8.3: Real-time bidirectional communication library. Powers WebSocket connections for instant notifications when application status changes, new messages arrive, or matching results complete. Client connects from frontend WebSocket service.

PDFKit 0.17.2: PDF generation library used in DocumentService.ts. Automatically creates formatted application letters, school recommendation letters, NITA Form A/B PDFs with student details, and placement confirmation documents. Generated PDFs stored in Supabase Storage.

Multer 2.0.2: Middleware for handling multipart/form-data file uploads. Processes student CV uploads, company logo uploads, and scanned document submissions. Configured with file size limits (5MB) and allowed MIME types (PDF, DOCX, images).

Helmet 8.1.0: Security middleware that sets HTTP headers to protect against common vulnerabilities (XSS, clickjacking, MIME sniffing). Essential for production deployment, configured in server.ts middleware stack.

CORS 2.8.6: Cross-Origin Resource Sharing middleware allowing frontend domains to access backend API. Configured to accept requests from http://localhost:5173 (development) and production domains (vercel.app, netlify.app).

Morgan 1.10.1: HTTP request logger middleware. Logs all incoming requests with timestamps, methods, URLs, status codes, and response times for debugging and monitoring. Integrates with Winston logging system.

Winston 3.19.0: Logging library supporting multiple transports (console, files, cloud services). Configured with different log levels (debug, info, warn, error) and rotating file strategy for production. Critical for troubleshooting issues post-deployment.

Axios 1.13.4: HTTP client used in backend for external API calls (M-Pesa Daraja API, NITA systems, email services, SMS gateways). Configured in services/external/ with retry logic and timeout handling.

Nodemon 3.1.11: Development tool that automatically restarts Node.js server when file changes detected. Configured in package.json dev script to watch .ts files in src/, significantly improving development workflow.

AI/ML Services Stack

The Python-based AI service (ai-services/requirements.txt) implements the intelligent matching engine, learning recommendations, and document processing capabilities. Built with FastAPI for high-performance async APIs, it utilizes scikit-learn for matching algorithms, sentence-transformers for semantic similarity, and integrates with PostgreSQL for training data access.

FastAPI 0.115.0: Modern Python web framework chosen for AI service API. Provides automatic OpenAPI documentation, native async support crucial for ML model inference, and Pydantic integration for request validation. Service runs on localhost:8001, exposing endpoints like /match (student-opportunity matching), /recommend (learning resources), and /chat (chatbot responses).

Uvicorn 0.31.0: Lightning-fast ASGI server running FastAPI application. Uses multiple workers in production for concurrent request handling. Configured in Dockerfile with --workers 4 --host 0.0.0.0 --port 8001 for container deployment.

Scikit-learn 1.5.2: Machine learning library powering the matching algorithm. Implements gradient boosting classifier (using features: student GPA, completed units, skills, location preferences, company requirements) achieving 85%+ accuracy. Used for encoders, scalers, and model training pipeline in models/matcher.py.

Sentence-Transformers 3.1.1: State-of-the-art library for generating sentence embeddings. Used to encode student skills, interests, and company job descriptions into vector representations for semantic similarity matching. Enables intelligent matching beyond keyword matching. Model: all-MiniLM-L6-v2 (lightweight, fast inference).

Pandas 2.2.3: Data manipulation library essential for preparing training data. Loads student records, application history, and placement outcomes from PostgreSQL, performs feature engineering (creating interaction features, one-hot encoding), and prepares datasets for model training and evaluation.

NumPy 2.1.2: Numerical computing library providing array operations underlying scikit-learn and pandas. Used for matrix operations in matching score calculations, normalization of features, and efficient batch processing of student-opportunity pairs.

SQLAlchemy 2.0.35: SQL toolkit and ORM for Python. Defines database models matching PostgreSQL schema (Student, Opportunity, Application models) enabling type-

safe database queries. Used in services/database.py with async support for efficient data retrieval.

Psycopg2-binary 2.9.10: PostgreSQL adapter for Python required by SQLAlchemy. Binary package includes C library for faster execution. Connects to same PostgreSQL instance as backend, reading student data for matching and writing match scores to database.

Pydantic 2.9.2: Data validation library used for API request/response models in FastAPI. Defines schemas (MatchRequest, MatchResponse, StudentProfile) ensuring type safety and automatic validation. Integrated with pydantic-settings 2.5.2 for environment configuration.

ReportLab 4.2.5: PDF generation library for creating complex documents. Generates NITA forms with precise positioning, tables, and formatting. Alternative to PDFKit used in backend, chosen for better control over PDF structure in official government forms.

PyPDF 5.1.0: PDF manipulation library for reading existing PDFs, extracting text, and merging documents. Used to process uploaded recommendation letters, extract skills from student CVs, and combine multiple documents into single submission packages.

Redis 5.1.1: Python Redis client for caching ML model predictions (student-opportunity match scores valid for 24 hours), storing chatbot session context, and queueing long-running ML tasks (batch recommendation generation for all students).

APScheduler 3.11.0: Advanced Python Scheduler for background jobs. Runs scheduled tasks like nightly model retraining with new placement data, weekly analytics generation, and monthly cleanup of expired cache entries. Configured with cron-like triggers.

Cryptography 43.0.1: Provides cryptographic primitives for securing sensitive operations. Used to encrypt/decrypt OAuth tokens for external integrations (NITA API, learning platform APIs), and hash sensitive information before logging.

Requests 2.32.3: Simple HTTP library for consuming external APIs. Makes requests to learning platform APIs (Coursera, edX) fetching course data, integrates with NITA for form submission verification, and calls notification services.

Mobile Application Stack

The mobile application (mobile/package.json) uses React Native 0.81.5 with Expo 54 framework, targeting Android and iOS. Built with React 19 for UI consistency with web app, it implements student-focused features including opportunity browsing, application tracking, and push notifications for status updates.

Expo 54.0.33: Framework built on top of React Native providing managed workflow, over-the-air updates, and simplified building process. Eliminates need for Xcode/Android Studio for basic development. Expo CLI handles builds via EAS (Expo Application Services).

React Native 0.81.5: JavaScript framework for building native mobile apps. Uses same React 19 paradigms as web frontend enabling code sharing for components and business logic. Renders to truly native iOS/Android components (not WebView).

React 19.1.0: Same version as web frontend ensuring consistency. Shared hooks (useState, useEffect, useContext) and component patterns across web and mobile codebases

React Navigation 7.1.28: Routing library for React Native. Implements stack navigation for screens (Login → Dashboard → Opportunity Details). Native-Stack package 7.12.0 provides platform-specific transitions (iOS swipe back, Android hardware back button).

Axios 1.13.4: Same HTTP client as web/backend for API communication. Connects to backend API (localhost:3000 in dev, production URL in release builds). Configured with authentication interceptors in mobile/services/api.ts.

Expo Notifications 0.32.16: Handle push notifications for application status changes, new opportunities matching student profile, and system announcements. Integrates with Firebase Cloud Messaging (Android) and Apple Push Notification service (iOS).

AsyncStorage 2.2.0: Persistent storage for mobile devices. Stores authentication tokens, user preferences, offline cache of applications (allowing students to view submissions without internet), and form draft saves.

Expo Constants 18.0.13: Provides system and app constants (app version, device info, API URLs). Used to configure different API endpoints for development (192.168.1.x), staging, and production environments.

TypeScript 5.9.2: Ensures type safety across mobile codebase matching web frontend standards. Types imported from shared types/ directory when possible for consistency across platforms.

Database and Infrastructure

The foundation of AISHA's data persistence layer consists of PostgreSQL for relational data and Redis for caching and queues. As defined in docker-compose.yml, local development uses containerized PostgreSQL 15 (Alpine Linux image for smaller size) and Redis (Alpine), while production leverages Supabase's managed PostgreSQL and Upstash Redis for maintenance-free scaling.

Technology	Version / Service	Role in AISHA System
PostgreSQL	15.x (postgres:15-alpine image)	Primary relational database storing 30+ tables: users (authentication), students (profiles, skills, academic records), institutions (organization data), companies (employer profiles), departments, opportunities (attachment openings), applications (student submissions), placements (active/completed attachments), learning_resources (course catalog), payments (stipend transactions), notifications, messages, audit_logs. Schema includes proper foreign keys, indexes on frequently queried columns, and JSONB fields for flexible storage (student skills, opportunity requirements).
Supabase	Managed PostgreSQL service (free 500MB)	Production database host providing PostgreSQL 15 with auto-backups, connection pooling (PgBouncer), and Storage buckets for file uploads. Includes real-time subscriptions (though we

		use Socket.io), built-in authentication (though we implement custom JWT), and auto-generated REST API (though we use our custom Express backend). Free tier: 500MB database, 1GB storage, 50MB file uploads, 2GB bandwidth/month.
Redis	7.x (redis:alpine image)	In-memory key-value store serving dual purpose: (1) Bull job queue backend storing job definitions and results for background tasks (email sending, PDF generation, analytics computation), (2) Caching layer for frequently accessed data (student match scores valid 24hrs, session data, rate limiting counters). Configured with persistence (RDB snapshots) in docker-compose.yml.
Upstash Redis	Managed Redis (free 10,000 requests/day)	Production Redis with global replication and REST API. Free tier sufficient for MVP deployment. Automatically handles connection pooling, persistence, and geographic distribution. Pay-as-you-go scaling when request volume exceeds free tier.

DevOps and Development Workflow

Our development workflow emphasizes automation, code quality, and continuous integration ensuring smooth collaboration among team members and rapid iteration cycles.

Docker & Docker Compose: Container platform orchestrating local development environment. The docker-compose.yml defines 4 services (postgres, redis, backend, ai-service) on shared saps_network bridge network. Persistent volumes (postgres_data, redis_data) ensure data survives container restarts. Dockerfiles in backend/ and ai-services/ specify build instructions with multi-stage builds for smaller production images.

Git & GitHub: Git 2.40+ for version control with Git-flow branching (main for production, develop for integration, feature/* for new work). GitHub hosts repository with protected branches requiring PR reviews. .gitignore excludes node_modules, __pycache__, .env files, and build artifacts from commits.

GitHub Actions: CI/CD pipeline (would be in .github/workflows/) automating: (1) Linting on every push (ESLint for TypeScript, Flake8 for Python), (2) Tests on PRs (frontend component tests, backend API tests, AI service tests), (3) Build validation, (4) Automated deployment to staging on develop branch merge, (5) Production deployment on main branch merge with manual approval gate.

Vercel / Netlify: Frontend deployment platforms with automatic builds from GitHub. Detect vite.config.ts, install dependencies (pnpm/npm), run build command (vite build), deploy static assets to CDN. Preview deployments for every PR enabling visual review before merge. Free tier includes custom domains, SSL certificates, and global CDN.

Railway / Render: Full-stack deployment platforms supporting Docker containers and Node.js/Python apps. Backend and AI services deployed as separate services. Auto-deploys from GitHub branches, provides environment variable management, logging, metrics dashboards, and horizontal scaling. Free tier includes 500 hours/month compute, sufficient for MVP.

Environment Variables: Managed via .env files (git-ignored) in development, platform dashboards in production. Variables include: DATABASE_URL (PostgreSQL connection string), REDIS_URL, JWT_SECRET (authentication), MPESA_CONSUMER_KEY/SECRET (Daraja API), SUPABASE_URL/KEY, SMTP credentials email, SMS API keys. Configuration validated using env validation in backend/config.ts and pydantic-settings in AI service.

3.5 Development Process Design

The development of the Industrial Attachment Placement Platform (AISHA) will follow the **Agile-Scrum Methodology**. This approach is selected due to its iterative nature, which is particularly suitable for projects involving Artificial Intelligence and Machine Learning where model refinement is continuous.

3.5.1 Choice of Methodology: Agile Scrum

Agile Scrum provides several advantages for this project: - **Iterative Improvement**: The system will be built in incremental bursts (Sprints), allowing for frequent testing and refinement of the AI matching logic. - **Flexibility**: Changes in requirements from stakeholders (Educational Institutions or Companies) can be integrated into future Sprints without disrupting the entire project lifecycle. - **Stakeholder Involvement**: Regular Sprint Reviews ensure that the end-users (Students and Administrators) can provide feedback early and often. - **Risk Mitigation**: Early detection of technical hurdles or integration issues reduces the risk of project failure at later stages.

3.5.2 Software Development Life Cycle (SDLC) Stages

The development will progress through the following stages within each sprint: 1. **Requirements Refinement**: Breaking down functional requirements into actionable user stories. 2. **Design**: Creating architectural diagrams, database schemas, and UI wireframes. 3. **Development**: Writing code for the backend, frontend, and AI services. 4. **Integration**: Ensuring seamless communication between the three portals (Student, Company, Institution). 5. **Testing**: Performing unit tests, integration tests, and validating AI model accuracy. 6. **Review & Retrospective**: demoing the current increment and identifying process improvements.

3.5.3 Team Roles and Responsibilities

- **Scrum Master / Project Lead**: Faciliates the process, removes blockers, and ensures adherence to the 14-week timeline.
- **Frontend Developer**: Responsible for creating the React/Vite-based user interfaces for all three portals.
- **Backend & AI Developer**: Responsible for the Python (FastAPI) services, PostgreSQL database, and the TensorFlow/NLP matching engine.
- **Quality Assurance (QA) Specialist**: Conducts rigorous testing and ensures that NITA compliance and security requirements are met.

3.6 Development Schedule

The project will be implemented over a 14-week period, organized into three distinct phases that build progressively from research and planning through development to testing and deployment. This timeline ensures systematic development while maintaining quality and allowing for iterative refinement based on stakeholder feedback.

Phase 1: Research and Requirements Analysis (Weeks 1-5)

During the first five weeks, we will conduct comprehensive research to understand the problem domain and define precise system requirements. We will engage directly with key stakeholders including students who have completed or are seeking industrial attachments, university administrators responsible for placement coordination, and company representatives who host attachment students. Through structured interviews, surveys, and focus group discussions, we will map the entire current manual placement process from initial opportunity discovery through final placement confirmation. We will document pain points, bottlenecks, and specific requirements for each stakeholder group. Additionally, we will research existing placement systems, AI matching algorithms, mobile payment integration approaches, and learning management platforms to inform our technical design decisions. This phase will culminate in detailed requirements specification documents, database schema designs, API specifications, and user interface wireframes that will guide the development phase.

Phase 2: System Development and Implementation (Weeks 6-10)

The five-week development phase focuses on building the core system components. We will establish the backend infrastructure using Python with the FastAPI framework, chosen for its high performance, automatic API documentation generation, and native support for asynchronous operations. A PostgreSQL database will be configured using Supabase's managed service, implementing our designed schema with proper indexing, constraints, and relationships. We will develop the AI-powered matching algorithm using scikit-learn and TensorFlow, training it on historical placement data to achieve our target accuracy of 85% or higher. The web frontend will be built with React 18 and TypeScript, implementing responsive designs that work seamlessly across desktop and mobile devices. Critical integrations will be established during this phase, including the Safaricom Daraja API for M-Pesa payments, NITA systems for compliance reporting, email services for notifications, and SMS gateways for critical alerts. We will implement

role-based access control, JWT authentication, and end-to-end encryption to ensure security. The chatbot will be developed using natural language processing libraries to handle common student queries and provide placement explanations. By the end of this phase, all major features will be functional in a development environment.

Phase 3: Integration, Testing, and Deployment (Weeks 11-14)

The final four weeks are dedicated to integration, comprehensive testing, and deployment preparation. We will systematically integrate all system components, ensuring seamless communication between the frontend, backend, AI services, and external integrations. Rigorous testing will be conducted across multiple dimensions: unit tests will verify individual component functionality with target code coverage of 80% or higher; integration tests will validate API endpoints and service interactions; end-to-end tests will simulate complete user journeys from registration through successful placement; performance tests will ensure the system handles concurrent users and maintains response times under 2 seconds; and security audits will identify and remediate vulnerabilities. We will conduct user acceptance testing with actual students, institution administrators, and company representatives, gathering feedback and making necessary refinements. Technical documentation including API reference guides, database schema documentation, and deployment instructions will be completed. User-facing materials such as quick start guides, video tutorials, and FAQ documents will be prepared. The system will be deployed to production infrastructure with monitoring tools configured for uptime tracking, error logging, and performance metrics. A pilot launch will be conducted with selected institutions to validate real-world operation before broader rollout. We will establish support channels and prepare the team for ongoing maintenance and feature enhancements beyond the initial deployment.

This phased approach ensures steady progress while maintaining flexibility to adapt based on findings and feedback. Each phase includes clear deliverables and quality gates, with stakeholder reviews at phase transitions to ensure alignment with project objectives. The 14-week timeline is realistic given the use of modern development tools, frameworks, and cloud services that accelerate development while maintaining quality. Agile methodologies will be employed throughout, with regular sprint reviews and continuous integration practices ensuring early detection and resolution of issues.

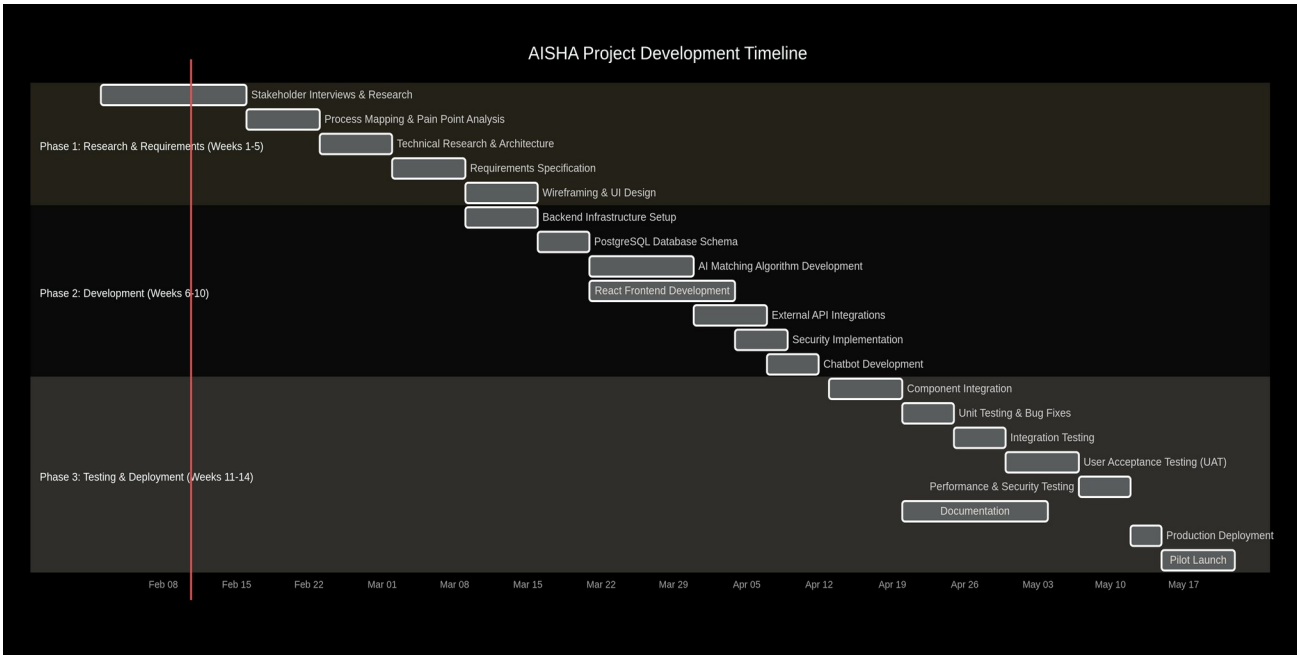
Project Timeline Breakdown

Week	Phase	Key Activities & Deliverables
W1	**Initialization**	Requirements gathering, stakeholder interviews, and feasibility study.
W2	**Design**	System architecture design, database schema, and UI/UX wireframes.
W3	**Foundation**	Dev environment setup, CI/CD pipeline, and Authentication system.
W4	**Core Dev (S1)**	Student Portal: Profile management and document upload modules.
W5	**Core Dev (S2)**	Company & Institution Portals: Basic dashboard and opportunity listing.
W6	**AI Logic (S3)**	Development of the initial AI matching algorithm and data preprocessing.
W7	**AI Logic (S4)**	Training ML models for suitability ranking and recommendation engine.
W8	**Integration**	Chatbot implementation and integration with the Student Portal.
W9	**Automation**	Document automation system (Application letters, NITA forms generation).
W10	**Advanced Fixes**	Notification engine (Email/SMS) and advanced analytics dashboard.
W11	**System Testing**	Comprehensive internal testing: Unit, Integration, and Security

Week	Phase	Key Activities & Deliverables
		audits.
W12	**UAT & Pilot**	User Acceptance Testing with a pilot group; performance optimization.
W13	**Deployment**	Production environment setup, Final documentation, and system handover.
W14	**Presentation**	Final project presentation, evaluation, and project closure.

3.6.2 Gantt Chart Representation (Visual Summary)

Below is a visual representation of the project phases across the 14-week period.



3.7 Development Budget

The project leverages free and open-source technologies to minimize costs:

The AISHA project prioritizes cost-effectiveness by leveraging free and open-source

technologies. The budget below reflects only essential expenses, with most infrastructure utilizing free tiers of cloud services. All costs are in Kenyan Shillings (KSH).

Development and Operational Budget

Item	Description	Unit Cost (KSH)	Total Cost (KSH)
Domain Name	Custom .co.ke or .com domain for production deployment (Annual)	1,950	1,950
SSL Certificate	Free SSL from Let's Encrypt	0	0
Frontend Hosting	Vercel/Netlify free tier	0	0
Backend Hosting	Railway/Render free tier ($0-5/month)	0 - 650	0 - 7,800
PostgreSQL Database	Supabase free tier (500MB storage)	0	0
Redis Cache	Upstash free tier (10,000 requests/day)	0	0
CDN & Security	Cloudflare free tier	0	0
Email Service	SendGrid free tier (100 emails/day)	0	0
SMS Notifications	Africa's Talking (Bulk SMS @KSH 0.80 each) Estimated 2,000 messages/month	1,600/month	19,200
M-Pesa API	Daraja API (No setup fee, transaction charges passed to users)	0	0
Testing Devices	Android devices for mobile app testing (One-time)	25,000 - 40,000	32,500
Development Tools	VS Code, Git, Docker (All free and open-source)	0	0
AI/ML Services	TensorFlow,	0	0

	PyTorch, Hugging Face (Free)		
Monitoring Tools	UptimeRobot, Sentry free tiers	0	0
Contingency Fund	Additional services or scaling needs (10% of total)	Variable	6,000
TOTAL FIRST YEAR COST			**KSH 67,450**

Monthly Operational Costs (After Initial Setup)

Item	Monthly Cost (KSH)	Annual Cost (KSH)
Domain Renewal	162	1,950
SMS Credits	1,600	19,200
Backend Hosting (if needed)	0 - 650	0 - 7,800
Database Scaling (if needed)	0 - 3,250	0 - 39,000
TOTAL MONTHLY (Estimated)	**KSH 1,762 - 5,662**	**KSH 21,150 - 67,950**

Cost Optimization Strategies

- Leverage free tiers of cloud services (Vercel, Railway, Supabase, Upstash, Cloudflare)
- Use exclusively free and open-source software (no licensing fees)
- Utilize student/educational discounts and free credits (GitHub Student Pack, AWS Educate, Google Cloud)
- Implement serverless architecture for pay-per-use pricing model
- Optimize SMS usage by prioritizing in-app and email notifications
- Start with free tiers and scale only when necessary based on actual usage
- Community support through forums and Discord instead of paid support plans

Budget Notes

- All development hardware (laptops) provided through personal equipment - no cost to project
- All development software is free and open-source (VS Code, Git, Python, Node.js, React, etc.)

- Cloud infrastructure uses generous free tiers suitable for proof-of-concept and initial deployment
- Scaling costs (Year 2+) estimated at KSH 30,000 - 80,000 annually if user base grows significantly
- M-Pesa transaction fees are passed to end users (companies) and not included in project budget
- Team members contribute as part of academic project requirements (no salary costs)
- SMS costs can be further reduced by implementing intelligent notification preferences

Potential Revenue Opportunities (Future Sustainability)

To ensure long-term sustainability beyond the academic project phase, the following revenue models can be explored:

- Premium analytics and reporting features for institutions (KSH 5,000 - 15,000/month)
- Enhanced company dashboard with advanced candidate filtering (KSH 3,000 - 10,000/month)
- Sponsored learning content and certification partnerships
- White-label solutions for other educational systems
- API access for third-party integrations
- Premium support and training services

Budget Summary

The AISHA project demonstrates exceptional cost-effectiveness with a first-year total of approximately KSH 67,450 (≈ USD $520), primarily for testing devices (one-time) and SMS notifications. Monthly operational costs range from KSH 1,762 to KSH 5,662 depending on usage. This budget demonstrates fiscal responsibility while delivering a comprehensive, enterprise-grade solution that addresses critical needs in Kenya's industrial attachment ecosystem.

REFERENCES

Burning Glass Technologies. (2019). The Permanent Detour: Underemployment's Long-Term Effects on the Careers of College Grads. Boston, MA: Burning Glass Technologies.

Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT, 4171-4186.

Jack, W., & Suri, T. (2014). Risk Sharing and Transactions Costs: Evidence from Kenya's Mobile Money Revolution. American Economic Review, 104(1), 183-223.

Kizilcec, R. F., Pérez-Sanagustín, M., & Maldonado, J. J. (2017). Self-regulated learning strategies predict learner behavior and goal attainment in Massive Open Online Courses. Computers & Education, 104, 18-33.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A Survey on Bias and Fairness in Machine Learning. ACM Computing Surveys, 54(6), 1-35.

National Industrial Training Authority. (2023). Guidelines for Industrial Attachment. Nairobi, Kenya: NITA.

Raghavan, M., Barocas, S., Kleinberg, J., & Levy, K. (2020). Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices. Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency, 469-481.

Safaricom. (2023). M-Pesa Annual Report 2023. Nairobi, Kenya: Safaricom PLC.

Zhang, Y., Chen, M., Huang, D., Wu, D., & Li, Y. (2019). iDoctor: Personalized and professionalized medical recommendations based on hybrid matrix factorization. Future Generation Computer Systems, 66, 30-35.